

**Shreya Ghorui**

Posted on Apr 20



LangChain From Scratch — A Complete Beginner's Guide (with Diagrams)

#ai #python #langchain #llm

Ever wondered how tools like **ChatPDF** or a "**Book my trip**" **AI assistant** actually work under the hood? 🤔

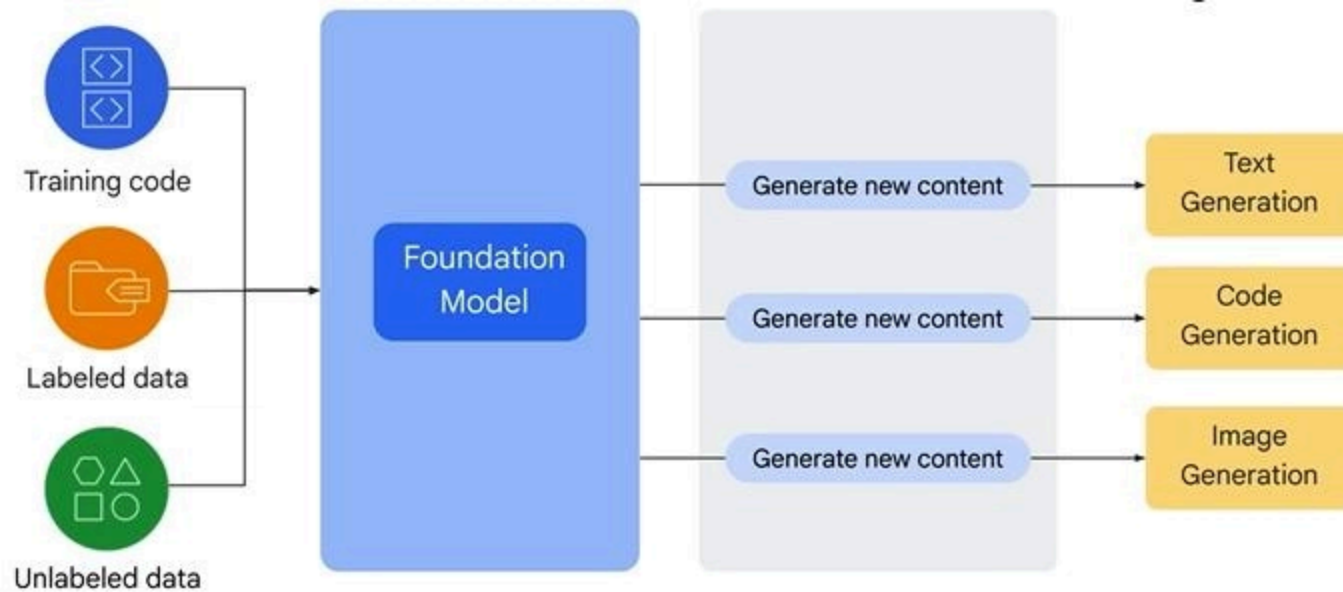
Behind most of them lies the same powerful framework: **LangChain** ⚡.

Imagine writing Python code that can read a 500-page PDF, understand your question about it, and reply like an expert — in seconds. That's LangChain.

Born as an open-source project to tame the complexity of building with LLMs, LangChain became the go-to framework for developers worldwide. And in this guide, you won't just read *about* it — you'll follow the data all the way through.

A Little Background — Foundation Models

Before diving into LangChain, it helps to understand the two perspectives people have when interacting with Foundation Models (like GPT-4, Claude, Gemini, etc.):



- **User Perspective** — You use products like ChatGPT, Claude.ai as an end user.
- **Builder Perspective** — You build applications *on top of* these models using APIs and frameworks.

LangChain is a tool for builders.

 **Quick Analogy:** Think of Foundation Models like a powerful industrial oven. A *user* just bakes bread in it. A *builder* designs the entire bakery — the recipes, the assembly line, the packaging — using that oven as the core engine. LangChain is your bakery blueprint.

📌 **What Happens Next?** We'll go through each of these modules one by one — from the very basics to building autonomous agents.

🤔 Why LangChain First?

LangChain is an open-source framework that helps in building LLM-based applications. It provides modular components and end-to-end tools that help developers build complex AI applications such as chatbots, question-answering systems, retrieval-augmented generation (RAG), autonomous agents, and more.

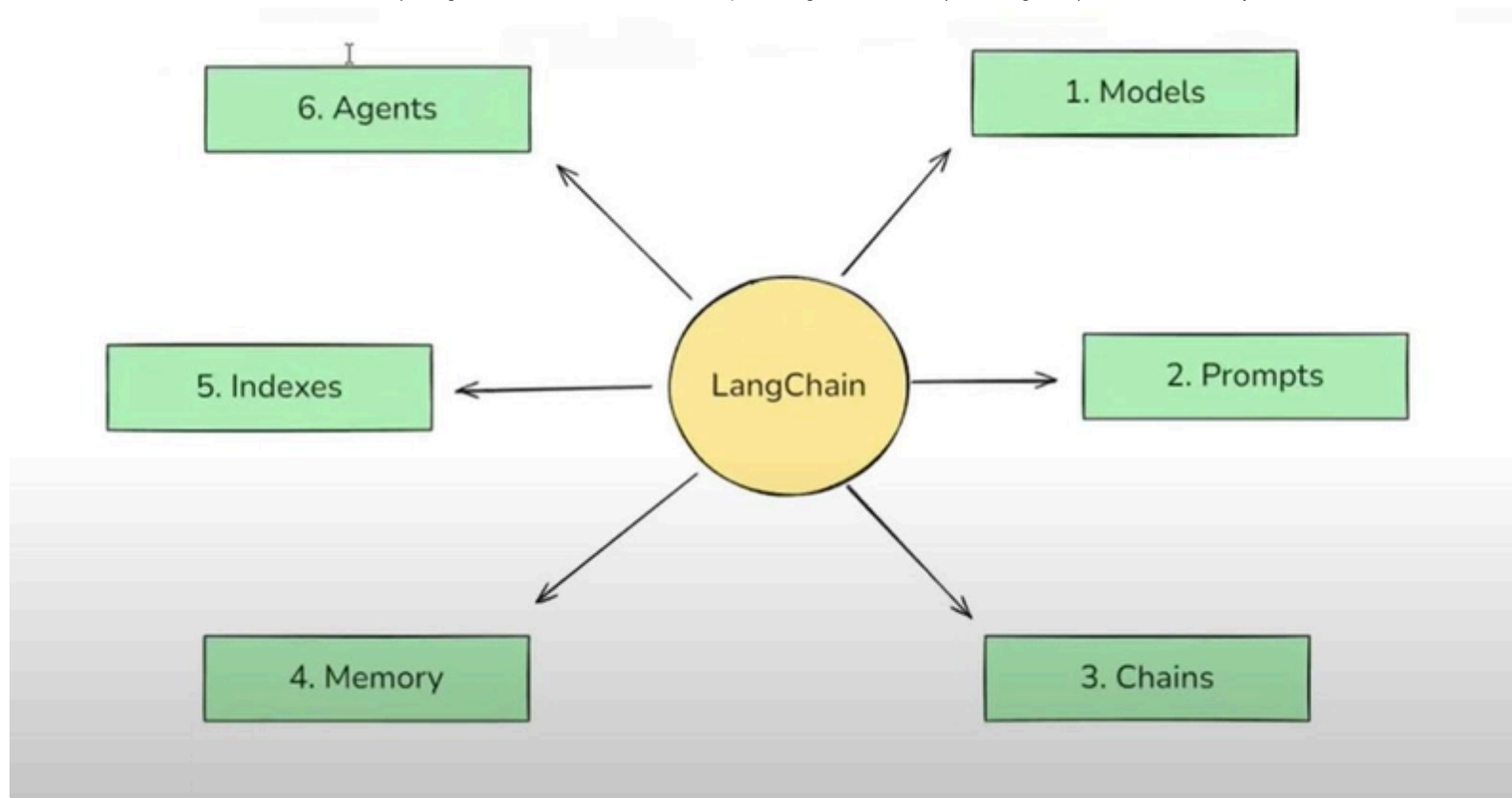
Key Benefits

1. Supports all the major LLMs
2. Simplifies developing LLM-based applications
3. Integrations available for all major tools
4. Open source / Free / Actively developed
5. Supports all major GenAI use cases

💡 One framework. Every model. Every use case. That's the LangChain promise.

🏗️ LangChain Components

LangChain is built around **6 core components** — think of them as the organs of a body, each with a specific job, all working together:



Let's open up each one.

1. 🤖 Models

➡ **What it is:** The interface to talk to any AI model.

In LangChain, *models* are the interfaces through which you interact with AI models.

The evolution of language models: NLP → NLU → LLMs → Internet scale (Billions of params, >100GB)

The Problem without LangChain:

Every provider (OpenAI, Anthropic, HuggingFace) has its own SDK, its own syntax, its own quirks. Switching models means rewriting your entire codebase.

The LangChain Solution:

A single, unified `model.invoke()` interface — regardless of the provider.

```
# OpenAI via LangChain
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()
model = ChatOpenAI(model='gpt-4', temperature=0)
result = model.invoke("How divide the result by 1.5?")
print(result.content)
```

```
# Anthropic Claude via LangChain – same interface, different model!
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv

load_dotenv()
model = ChatAnthropic(model='claude-3-opus-20240229')
result = model.invoke("Hi who are you")
print(result.content)
```

💡 **Key idea:** Swap `ChatOpenAI` for `ChatAnthropic` and everything else stays the same. That's model-agnostic development.

📌 **What Happens Next?** Once you have a model, you need to talk to it intelligently. That's where **Prompts** come in.

2. 💬 Prompts

➔ **What it is:** Reusable, dynamic templates for talking to LLMs.

LLMs take **input** → **prompt** → **output**. A raw string works, but it's fragile. LangChain makes prompt management powerful, reusable, and structured.

🔍 **Quick Analogy:** A raw string prompt is like shouting an order at a chef. A `PromptTemplate` is like handing them a proper recipe card — structured, consistent, and repeatable every time.

➔ Step 1: Dynamic & Reusable Prompts

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template('Summarize {topic} in {emotion} tone')
print(prompt.format(topic='Cricket', length='fun'))
```

➔ Step 2: Role-Based Prompts

Give your LLM a persona — like a Doctor, a Lawyer, or a Code Reviewer:

```
chat_prompt = ChatPromptTemplate.from_template([
    ("system", "Hi you are a experienced {profession}"),
    ("user", "Tell me about {topic}"),
])
```

```
formatted_messages = chat_prompt.format_messages(  
    profession="Doctor",  
    topic="Viral Fever"  
)
```

➡ Step 3: Few Shot Prompting

Teach the model by example — show it what "good output" looks like before asking your real question:

```
examples = [  
    {"input": "I was charged twice for my subscription this month.", "output": "Billing Issue"},  
    {"input": "The app crashes every time I try to log in.", "output": "Technical Problem"},  
    {"input": "Can you explain how to upgrade my plan?", "output": "General Inquiry"},  
    {"input": "I need a refund for a payment I didn't authorize.", "output": "Billing Issue"},  
]  
  
example_template = """  
Ticket: {input}  
Category: {output}  
"""  
  
few_shot_prompt = FewShotPromptTemplate(  
    examples=examples,  
    example_prompt=PromptTemplate(  
        input_variables=["input", "output"],  
        template=example_template  
    ),  
    prefix="Classify the following customer support tickets into one of the categories: "  
        "'Billing Issue', 'Technical Problem', or 'General Inquiry'.\n\n",  
    suffix="User_input:\nCategory:",
```

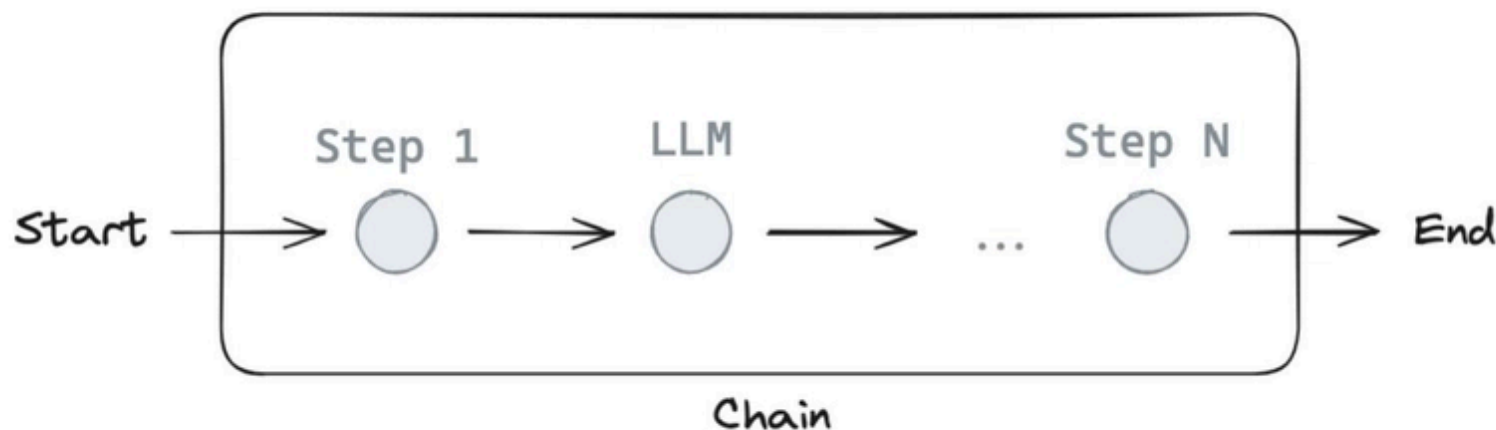
```
input_variables=["user_input"],  
)
```

📌 **What Happens Next?** Now that we can talk to models with structured prompts, we need to *connect* multiple steps together. That's what **Chains** do.

3. 🎯 Chains

➡ **What it is:** Pipelines that connect LLMs with other components.

Chains = **Pipelines**. They are the heart of LangChain (hence the name!). Instead of calling a model once, you chain multiple calls and operations together into a workflow.

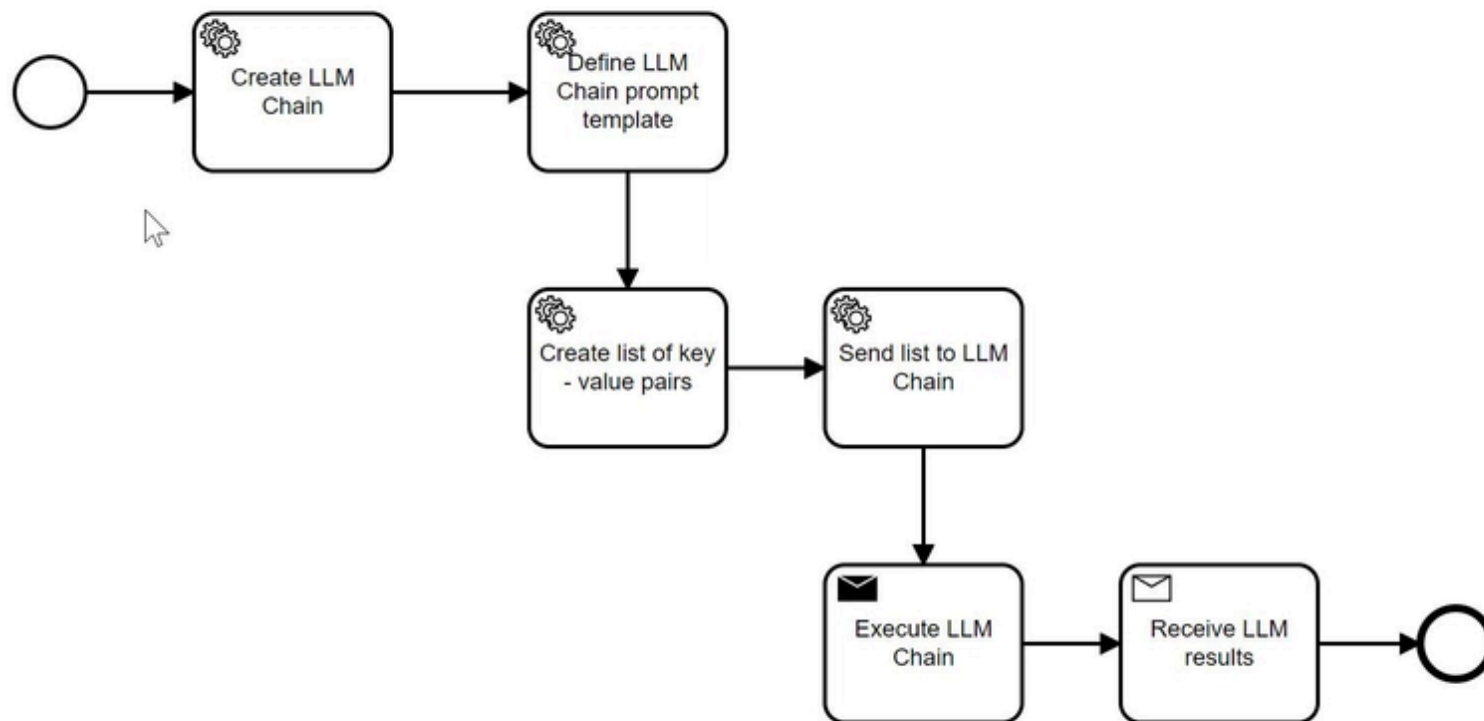


Quick Analogy: A single LLM call is like one chef making one dish. A Chain is the entire restaurant kitchen — prep cook → head chef → plating station — each step feeding the next.

Types of Chains

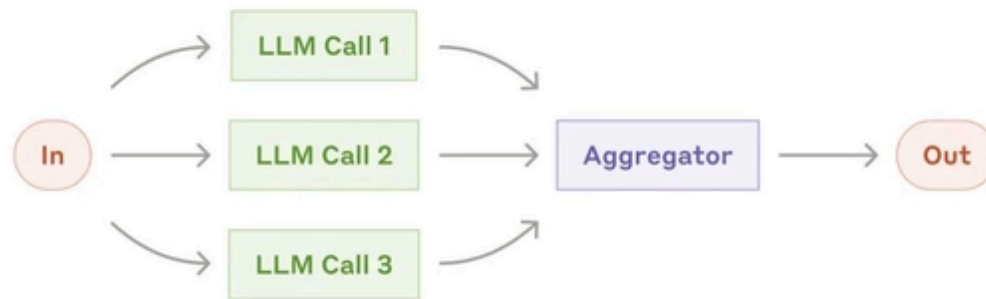
Sequential Chains — Steps run one after another:

Example: Translate 1000-word English text → Hindi summary (100 words)



Parallel Chains — Multiple LLMs run simultaneously, results combined:

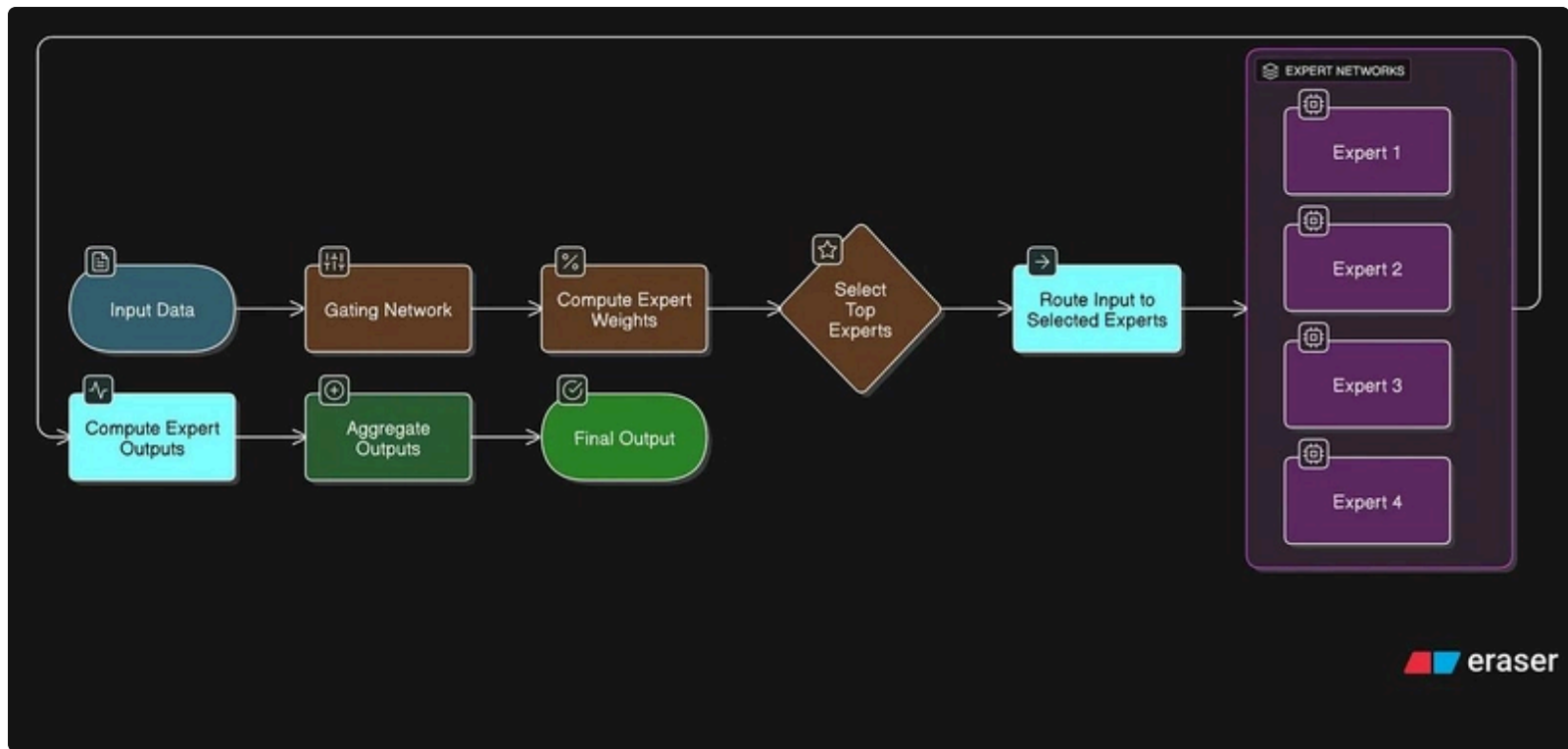
Example: Generate a report from two expert LLMs simultaneously, then merge



The parallelization workflow

● **Conditional Chains** — Route based on output:

Example: AI feedback agent — good feedback → "Thank you!", bad feedback → send email alert



📌 **What Happens Next?** Chains are stateless — they don't remember previous conversations. To build a real chatbot, we need **Memory**.

4. 🧠 Memory

➡ **What it is:** Giving your LangChain app the ability to remember.

Without memory, every API call is stateless — like talking to someone with amnesia who forgets you the moment you stop speaking.

LangChain's memory components let you **persist and retrieve conversation history**, making chatbots feel natural and context-aware.

 **Quick Analogy:** Memory is like a notepad your assistant keeps on the desk. Every time you talk, they jot down what was said. Next time you walk in, they already know your name, your preferences, and what you discussed last week.

 **What Happens Next?** Memory handles conversation history — but what if your app needs to search through *thousands of your own documents*? That's where **Indexes (RAG)** come in.

5. Indexes — The Power of RAG

 **What it is:** Connecting your LLM to external knowledge.

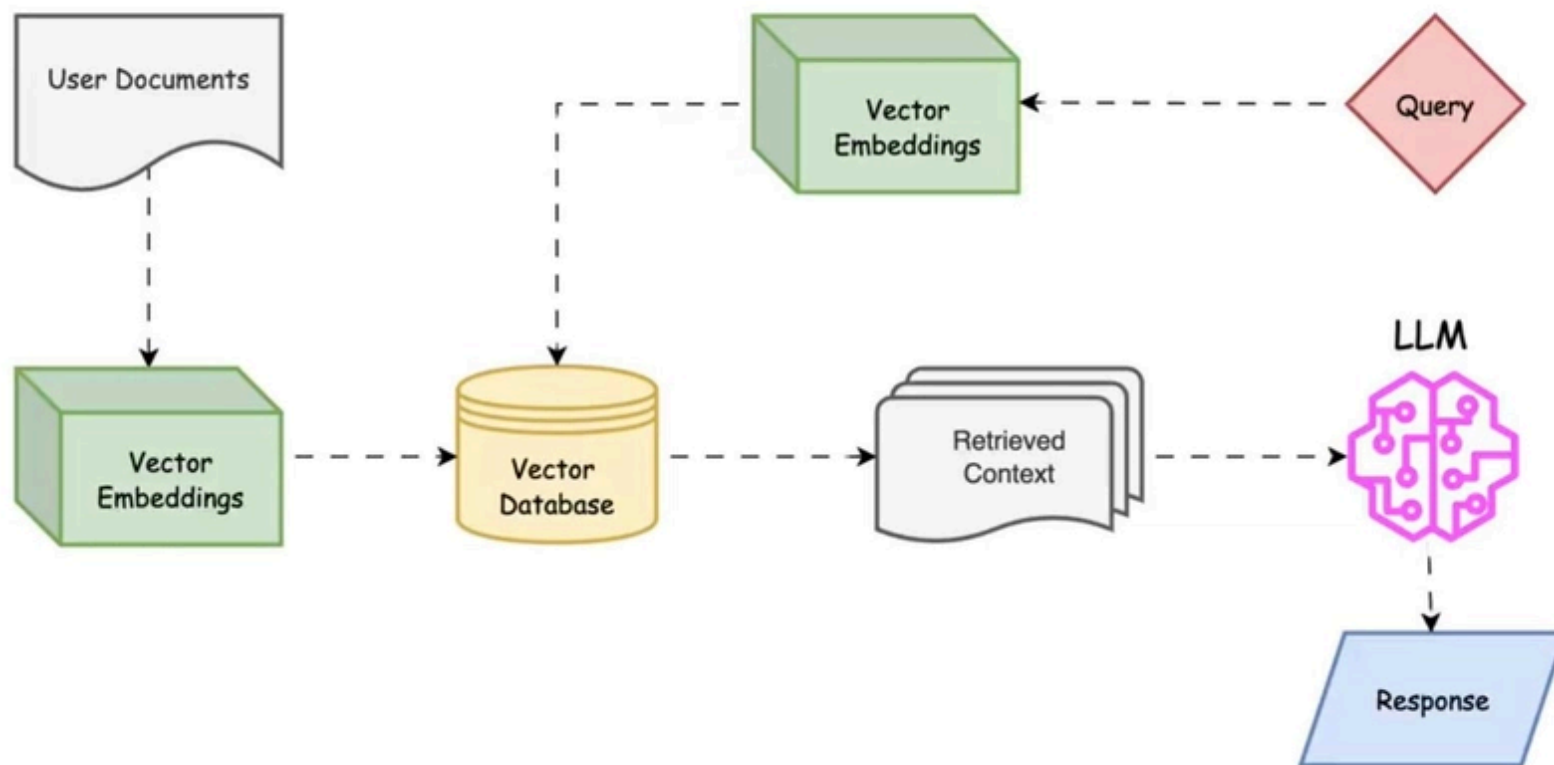
"Indexes connect your application to external knowledge — such as PDFs, websites, or databases."

This is the foundation of **RAG (Retrieval Augmented Generation)** — the most powerful pattern in modern AI apps.

The Problem: LLMs are trained on general internet data. They know nothing about *your* company's internal documents, *your* codebase, or *your* PDF notes.

The RAG Solution: Don't fine-tune the model — just give it your documents at query time.

 **Step 1: The Full RAG Pipeline**

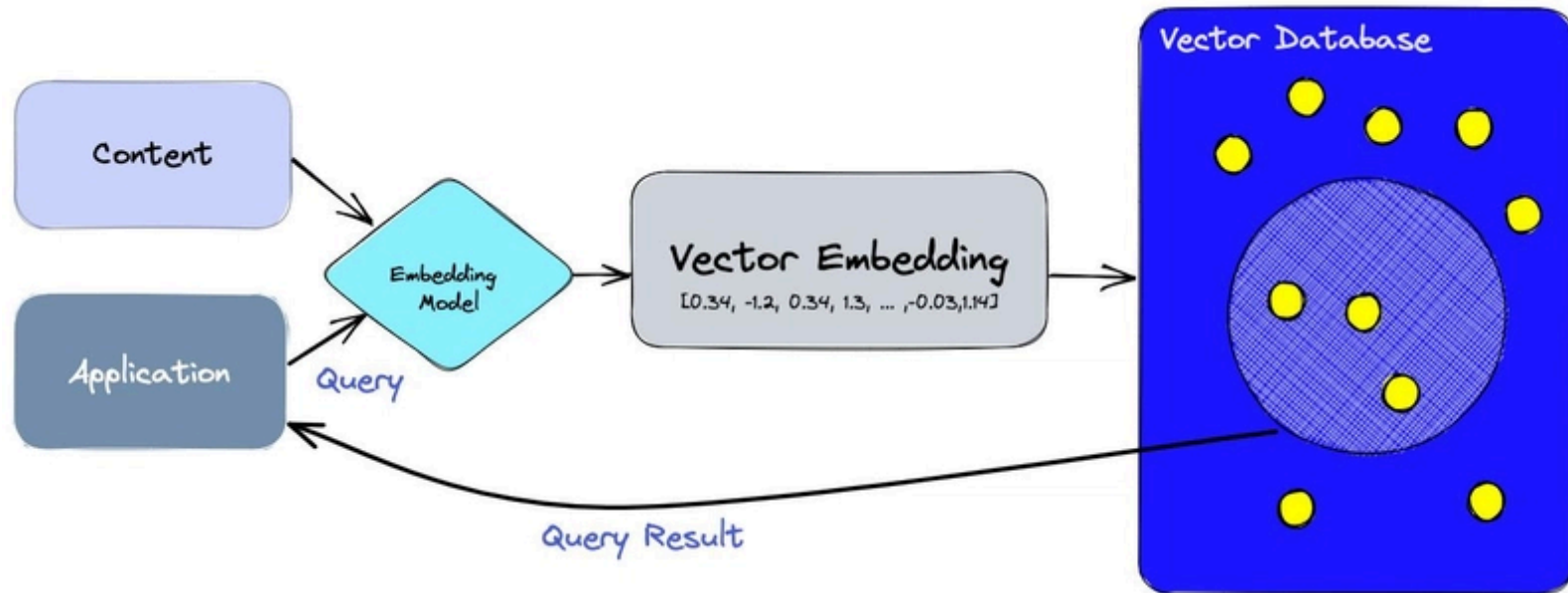


Here's what's happening step by step:

➔ Step 2: Understanding Embeddings & Semantic Search

Traditional search: keyword matching → "Virat" → [372, 961] (just index positions)

Semantic search converts text into **vectors** — high-dimensional numbers that capture *meaning*, not just spelling.



💡 **Key insight:** "How many runs?" and "total score of" mean the same thing — semantic search finds both. Keyword search finds neither unless the exact word matches.

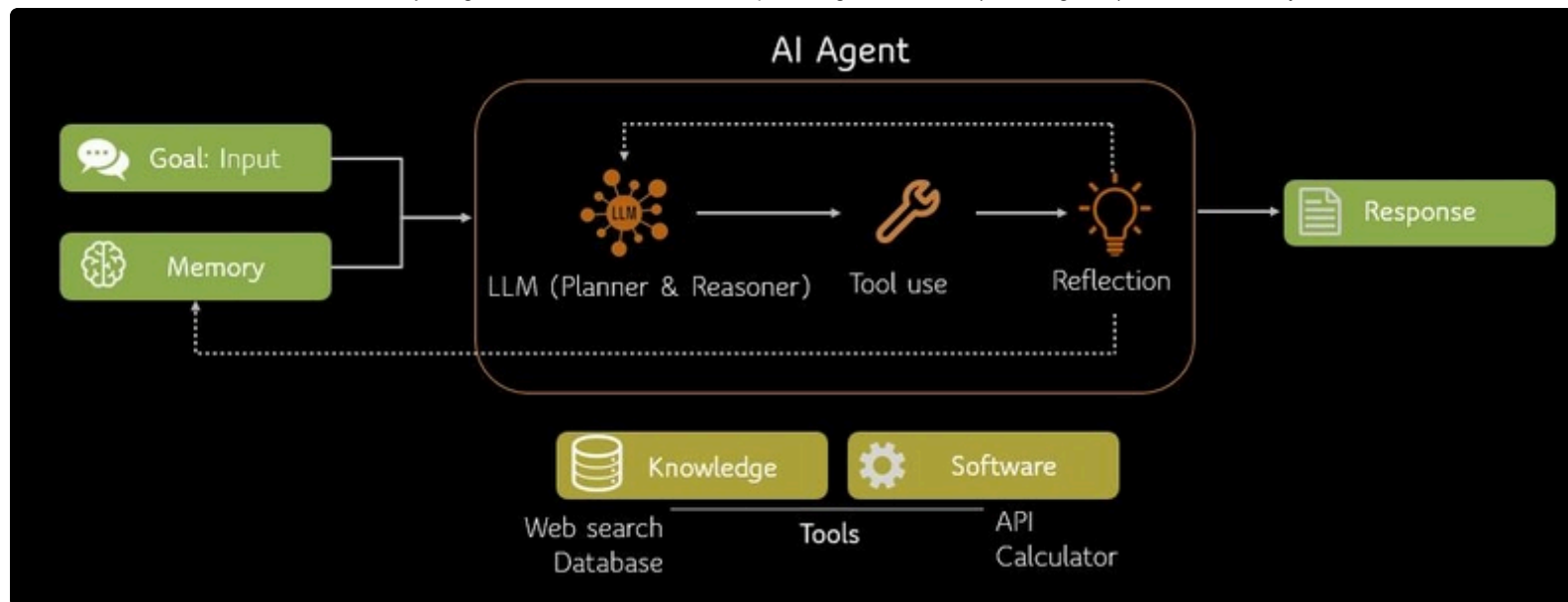
📌 **What Happens Next?** RAG gives your app access to documents. But what if you want your app to *act* — search the web, call an API, book a flight? That's what **Agents** do.

6. 🤖 Agents

➡ **What it is:** LLMs that can think, plan, and use tools.

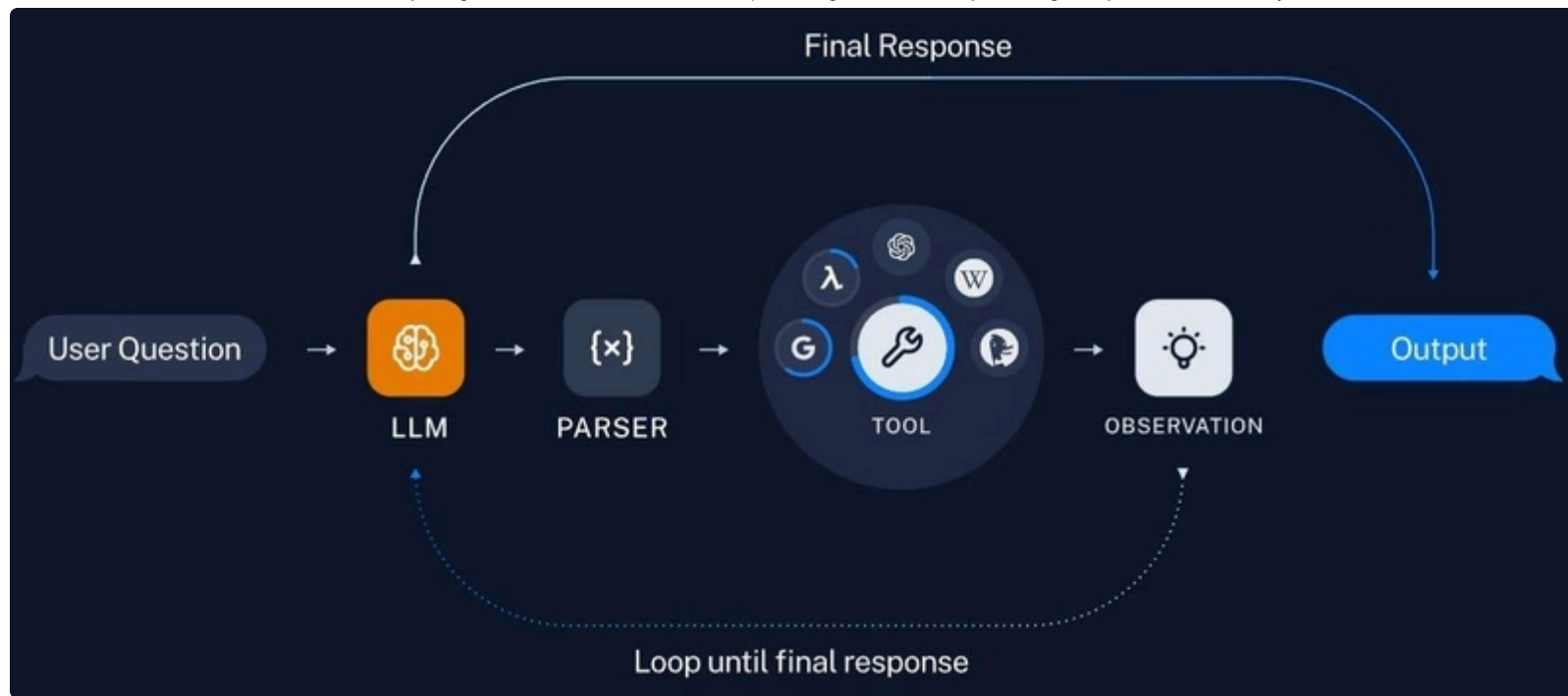
Agents are **AI systems** that combine:

- 🧠 **Reasoning capabilities** (the LLM brain — chain of thought)
- 🔑 **Tools** (external actions it can call)



Quick Analogy: A chatbot is like a very knowledgeable librarian — they can answer questions from memory. An AI Agent is like a personal assistant with a phone — they can answer questions *and* actually call the airline, book the hotel, and send you a confirmation.

How Agents Work



→ Step-by-Step Example

"Can you multiply today's temperature in Delhi with 3?"

Step 1: Agent reasons — "I need Delhi's current temperature. I have a Weather API tool."

Step 2: Agent calls **Weather API** → gets Delhi temp: **32°C**

Step 3: Agent reasons — "Now I need to multiply 32×3 . I have a Calculator tool."

Step 4: Agent calls **Calculator** → **96**

Step 5: Agent returns: "Today's temperature in Delhi is 32°C. Multiplied by 3 = **96." ✓

No hardcoding. No manual steps. Pure autonomous reasoning.

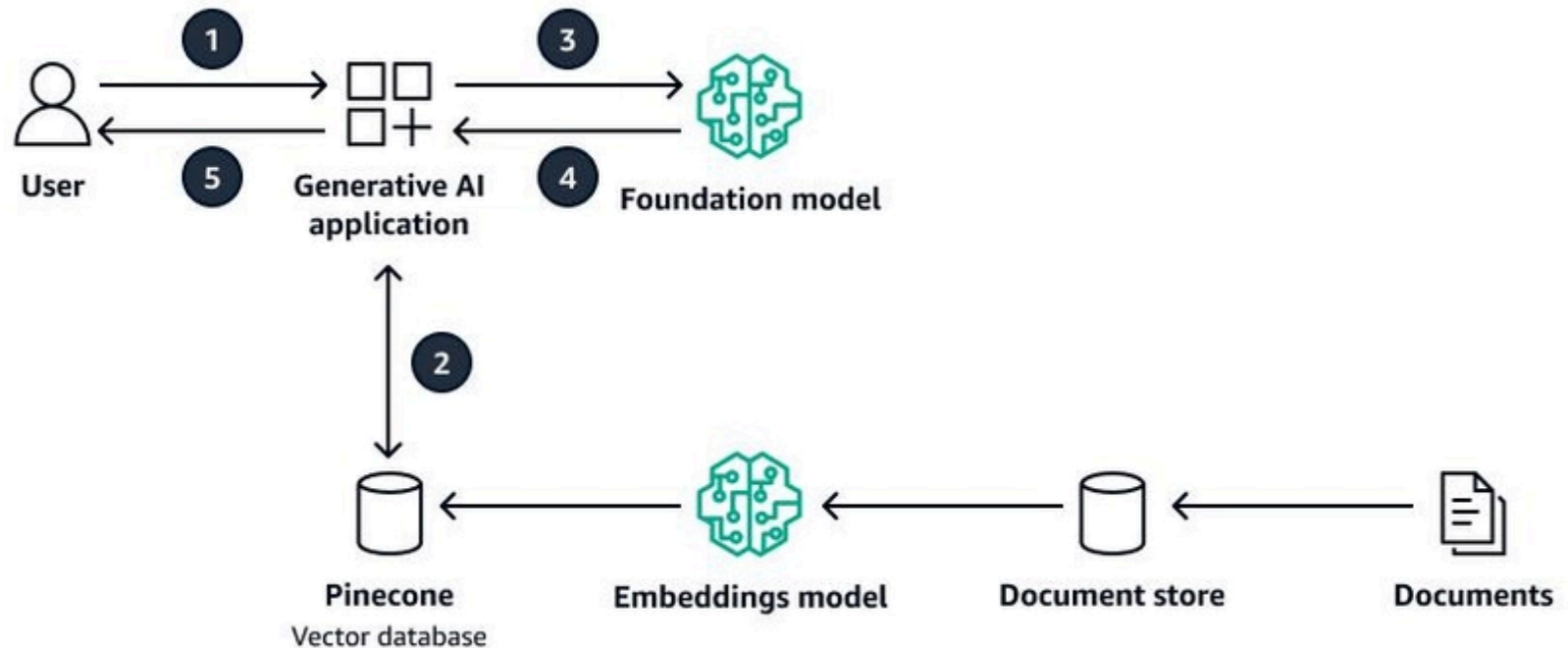
🔗 **What Happens Next?** Now that you know all 6 components, let's see what you can actually build with them!

What Can You Build with LangChain?

Application Type	Real Example
Conversational Chatbots	Scalable customer support bot that handles 10,000 queries/day
AI Knowledge Assistants	Q&A over your company's 500-page internal docs
AI Agents	"Make my trip" — searches flights, books hotels, sends itinerary
Workflow Automation	Multi-step pipelines: scrape → summarize → email → log
Summarization/Research Helpers	ChatPDF, research paper summarizer, legal doc analyzer

The Full Picture — Real-World RAG Architecture

Putting all 6 components together, here's how a production-grade LangChain RAG application works end-to-end:



Every component plays its role:

- **Models** → the brain (Google, OpenAI, Claude — swap anytime)
- **Prompts** → structured instructions sent to the brain
- **Chains** → the assembly line connecting every step
- **Memory** → remembers your conversation history
- **Indexes** → connects your PDFs/docs to the pipeline
- **Agents** → makes decisions and calls tools autonomously

Key Takeaways

1.  **LangChain = Framework** for building LLM apps, not an LLM itself
2.  **6 Components:** Models, Prompts, Chains, Memory, Indexes, Agents

3. 📖 **RAG** is the most powerful pattern for giving LLMs access to your data
4. 🤖 **Agents** = LLMs + Tools + Reasoning = true AI automation
5. 🔄 **Model Agnostic** — works with OpenAI, Anthropic, HuggingFace, Ollama, and more

🔑 Getting Started — Your First LangChain App

```
pip install langchain langchain-openai langchain-anthropic python-dotenv
```

```
# Your first LangChain chain – prompt + model piped together
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate

model = ChatOpenAI(model="gpt-4o-mini")
prompt = PromptTemplate.from_template("Tell me a fun fact about {topic}")

# The | operator chains prompt → model
chain = prompt | model
result = chain.invoke({"topic": "LangChain"})
print(result.content)
```

3 lines. One chain. That's the power of LangChain.

🌟 Conclusion: LangChain, Demystified

You've just walked through all 6 components of LangChain — from talking to models, to building RAG pipelines, to deploying autonomous agents.

No magic. No mystery. Just smart design:

- **Models** that unify every LLM behind one interface
- **Prompts** that make your instructions reusable and structured
- **Chains** that wire everything into a workflow
- **Memory** that makes your app feel human
- **Indexes** that connect your app to the real world
- **Agents** that think, plan, and act on their own

What started as a question — "*How do I build an LLM app?*" — became components, then pipelines, then autonomous systems.

And you?

You didn't just read about it. You followed the data all the way through. 🏆

Now go build something. 🚀

Drop your questions or project ideas in the comments — what are you planning to build with LangChain?

Tags: #langchain #llm #ai #python #machinelearning #generativeai



The DEV Team

PROMOTED





Gemini 3.5 Flash Developer Guide ⚡

Gemini 3.5 Flash is generally available (GA), stable, and ready for scaled production use. As our most

[Read More](#)

Top comments (2)



Anik Chand • Apr 20



Great thinking...



Some comments may only be visible to logged-in visitors. [Sign in](#) to view all comments.

[Code of Conduct](#) • [Report abuse](#)



Bitrise

PROMOTED



The CI/CD built for iOS. Ready for what's next

Build faster.
Ship with confidence.

See it in action today





Shreya Ghorui

Hi, I'm Shreya Ghorui, a B.Tech Information Technology student at Netaji Subhash Engineering College. I'm passionate about coding, love tackling new problems and have a growing interest in AI/ML and DS

LOCATION

Kolkata, West Bengal

EDUCATION

Netaji Subhash Engineering College

PRONOUNS

She/Her

JOINED

Jul 11, 2025

[Start a free trial](#)

Trending on DEV Community 🔥



Banning ChatGPT Won't Fix Your Shadow AI Problem. It'll Just Hide It.

#ai #security #devops #discuss



Anyone heard of Qoder?

#ai #programming #productivity #career



"Unlimited context" is not a feature. It's technical debt with better marketing.

#ai #architecture #programming #discuss

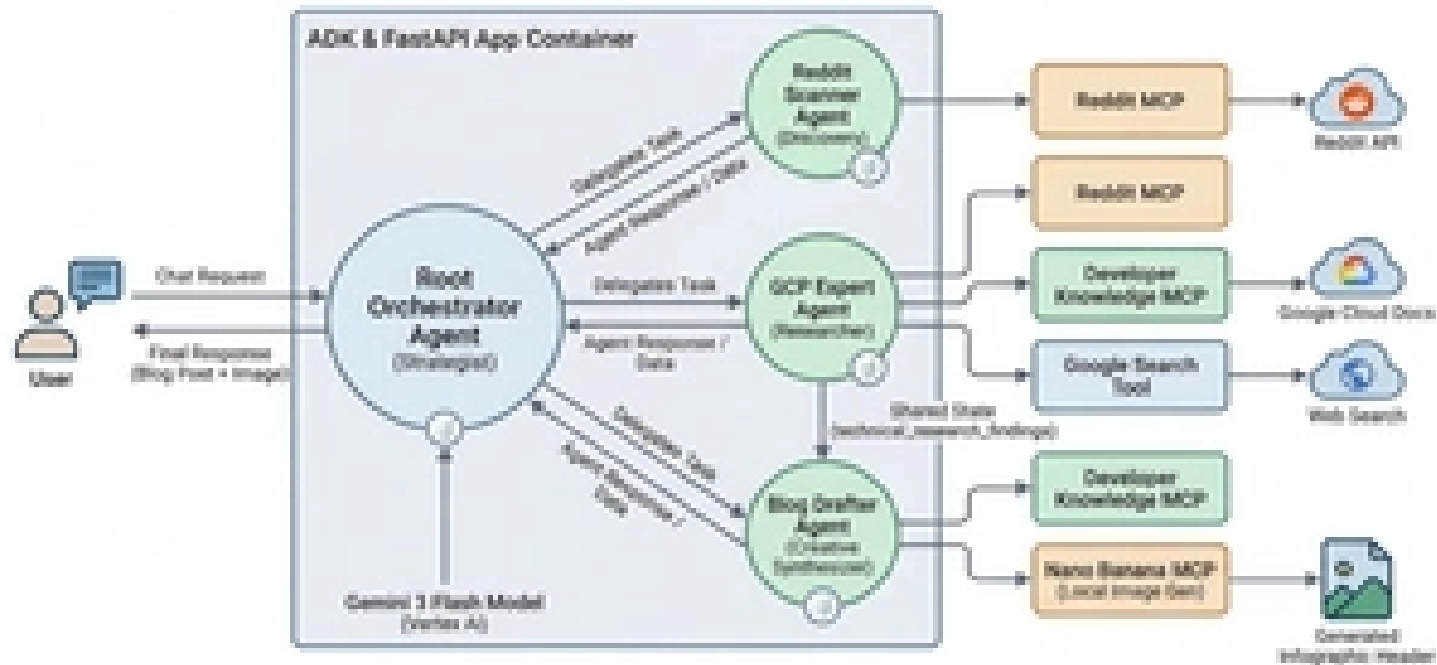


The DEV Team

PROMOTED

...

Dev Signal Multi-Agent Architecture (ADK & MCP)



Architect A Personalized Multi-Agent System with Long-Term Memory

In support of our mission to accelerate the developer journey on Google Cloud, we built Dev Signal — a multi-agent system designed to transform raw community signals into reliable technical guidance by automating the path from discovery to expert creation.

[Read more →](#)